Observaciones
Laboratorio 9

ISIS1225 Estructuras de Datos y Algoritmos, periodo 2025-20, grupo G06

Integrante	Código	Correo Uniandes
Nicolás Fajardo Aguirre	202413083	n.fajardoa@uniandes.edu.co
Christian Bravo	202322012	ca.bravom@uniandes.edu.co
Dennys Lozano	202413859	da.lozanom1@uniandes.edu.co

1. Análisis de los datos (Singapore Bus Routes)

Cada registro del archivo *bus_routes* corresponde a una parada de una ruta en una dirección. Por esta razón, para construir la MinPQ tomamos únicamente el registro con StopSequence igual a 1 de cada combinación (ServiceNo, Direction) y utilizamos su WD_FirstBus como prioridad. Al revisar el archivo más grande (*bus_routes_14000.csv*), encontramos lo siguiente:

Dato	Valor	Cómo lo manejamos
Registros (paradas)	13 903	Todos los guardamos en la lista stops, que es un array_list.
Registros con StopSequence igual a 1	379	Todos entran a la cola, así que la MinPQ queda con 379 elementos.
Combinaciones únicas (ServiceNo, Direction)	378	La ruta 410 en dirección 1 aparece dos veces. Sin embargo, desde la parada 3 sus horarios difieren (05:50 y 05:51), por lo que parecen dos variantes de la ruta y no un error; por eso insertamos ambas.
Rutas sin hora en WD_FirstBus	9	Estas rutas no operan entre semana (14A, 21A, 1N, 2N, 3N, 188R, 138A y 401) y el archivo trae un guion en lugar de la hora. Como la prioridad se compara como texto, ese símbolo queda antes que cualquier hora y, por lo tanto, estas rutas salen primero.
Primera ruta en salir	138A, dirección 1 (sin hora)	Después de las 9 rutas sin hora sale la 112A (00:15) y luego siguen las rutas de las 05:00.

Además, seguimos la indicación del laboratorio y usamos el valor de WD_FirstBus tal como viene en el archivo, es decir, como texto. Dado que todas las horas tienen cuatro dígitos (por ejemplo 0015 o 0500), compararlas como texto da el mismo orden que compararlas como números. Sin embargo, el guion de las rutas que no operan entre semana queda antes que cualquier hora, por lo que esas rutas salen de primeras aunque en realidad no tienen servicio en días laborales.

2. Preguntas de observación

a) ¿Qué estructura de datos subyacente se utiliza comúnmente para implementar una cola de prioridad en Python?

Comúnmente se utiliza un heap binario, también llamado montículo. Se trata de un árbol binario completo en el que cada padre tiene igual o más prioridad que sus hijos; en consecuencia, el elemento más prioritario siempre se encuentra en la raíz. Como el árbol es completo, puede guardarse en un arreglo sin apuntadores. En nuestra implementación, ese arreglo es un array_list del curso cuya posición 0 queda vacía (None); así, el padre de la posición *i* está en *i // 2*, mientras que sus hijos están en *2i* y *2i + 1*.

b) ¿Cuál es la diferencia entre una cola FIFO tradicional y una cola de prioridad?

En una cola FIFO el orden de atención coincide con el orden de llegada, es decir, sale primero el elemento que entró primero. En cambio, en una cola de prioridad sale primero el elemento con mejor prioridad, sin importar cuándo llegó. Por ejemplo, en el laboratorio la ruta 138A es la primera en atenderse, aunque en el archivo es la ruta número 84 en aparecer; por lo tanto, una cola FIFO la habría atendido en el puesto 84.

c) ¿Qué módulo proporciona Python para trabajar fácilmente con colas de prioridad?

Python ofrece el módulo heapq de la librería estándar, el cual convierte una lista en un montículo de mínimos mediante funciones como *heappush*, *heappop* y *heapify*. Asimismo, existe la clase queue.PriorityQueue, que utiliza heapq internamente y además agrega sincronización para trabajar con varios hilos. No obstante, en este laboratorio no utilizamos ninguno de los dos, ya que implementamos nuestra propia cola sobre las estructuras del curso.

d) ¿Qué ventajas tiene el uso de una cola de prioridad sobre una lista ordenada manualmente?

Una lista ordenada debe mantener todos sus elementos en orden, de manera que cada inserción busca la posición correcta y desplaza elementos, lo cual cuesta O(n). Por el contrario, el heap solo garantiza que la raíz sea el mejor elemento y, gracias a ello, insertar y retirar cuestan O(log n). Adicionalmente, la cola evita los errores de mantener el orden a mano y separa la operación que se necesita de la forma en que se implementa. Para comprobarlo, medimos con nuestras propias estructuras el tiempo de insertar n prioridades aleatorias y luego retirarlas todas:

n elementos	MinPQ (heap)	array_list ordenado a mano	Veces más rápido el heap
1 000	9 ms	48 ms	5
5 000	49 ms	1 104 ms	23
20 000	231 ms	18 239 ms	79

Como se observa, la diferencia crece a medida que aumenta n, lo cual es coherente con comparar un costo total de O(n log n) frente a uno de $O(n^2)$.

e) Si dos elementos tienen la misma prioridad, ¿cómo decide la cola cuál atender primero?

El heap no garantiza ningún orden entre elementos con igual prioridad, es decir, no es estable. Por consiguiente, el elemento que sale primero depende de la posición que ocupó cada uno en el árbol después de las operaciones swim y sink, y el enunciado acepta ese comportamiento. De hecho, lo observamos con el ejemplo del enunciado: las rutas 12 y 14 salen ambas a las 05:30 y la cola entregó primero la 14, aunque la 12 había llegado antes. Si se necesitara desempatar por orden de llegada, bastaría con agregar un segundo criterio a la prioridad, por ejemplo la pareja (prioridad, número de llegada).

f) ¿Qué situaciones del mundo real se pueden modelar con colas de prioridad? Mencione al menos dos.

En primer lugar, la planificación de procesos en un sistema operativo, donde se ejecuta primero el proceso de mayor prioridad. En segundo lugar, el cálculo de rutas más cortas con el algoritmo de Dijkstra, que usan los GPS y las redes de comunicación, ya que en cada paso se expande el nodo con menor distancia acumulada. Asimismo, el despacho de transporte, como en este laboratorio, permite ordenar las rutas según la hora de salida del primer bus. Finalmente, las simulaciones por eventos y las colas de impresión o de red también se modelan así, puesto que se procesa primero el evento más próximo o el trabajo más urgente.

g) En un sistema de atención médica, ¿cómo se puede usar una cola de prioridad para organizar a los pacientes?

Una cola de prioridad permite implementar el triage. Para ello, cada paciente entra a una MinPQ con su nivel de urgencia como prioridad (desde 1, reanimación, hasta 5, no urgente) y con sus datos como valor. De esta forma, el médico que queda libre ejecuta *remove()* y atiende al paciente más urgente, sin importar quién llegó antes. Por otra parte, si un paciente empeora mientras espera, *improve_priority()* le sube la prioridad. Por último, para que entre dos pacientes con el mismo nivel se atienda primero al que llegó antes, la prioridad puede definirse como la pareja (nivel, hora de llegada).

h) ¿Cómo afectaría al rendimiento usar una lista simple en lugar de una estructura especializada como heap_pq para manejar prioridades?

Con una lista simple, una de las dos operaciones principales queda en O(n). Por un lado, si la lista no está ordenada, insertar cuesta O(1), pero atender obliga a recorrer toda la lista para hallar el mínimo, lo cual cuesta O(n). Por otro lado, si la lista está ordenada, atender cuesta O(1), mientras que insertar cuesta O(n) porque hay que buscar la posición y desplazar elementos. En consecuencia, con n operaciones de cada tipo el costo total pasa de O(n log n) con el heap a $O(n^2)$. Con los 379 elementos del archivo la diferencia casi no se nota; sin embargo, la tabla de la pregunta d) muestra que con 20 000 elementos la lista ordenada ya es unas 79 veces más lenta.

i) ¿Qué complejidad temporal tiene la inserción y eliminación/atención en una cola de prioridad basada en heap?

La altura de un heap con n elementos es $\log_2(n)$ y, como swim y sink recorren a lo sumo un camino entre la raíz y una hoja, tanto la inserción como la eliminación cuestan O(log n). La siguiente tabla resume la complejidad de cada operación en nuestra implementación:

Operación	**Complejidad**	**Motivo**
insert	O(log n)	Agrega el elemento al final y luego hace swim.
remove (atender)	O(log n)	Mueve el último elemento a la raíz y luego hace sink.
get_first_priority	O(1)	Lee la raíz, que está en la posición 1.
size, is_empty	O(1)	El heap lleva un contador de elementos.
is_present_value, contains	O(n)	El heap no está ordenado para buscar, por lo que hay que recorrerlo.
improve_priority	O(n)	Requiere una búsqueda de costo O(n) y después un swim de costo O(log n).